# GITHUB

## Complete Course Notes

*Pluralsight | Git & GitHub Fundamentals — All Modules*

### Modules Covered

Module 02 — Overview of Git and GitHub
Module 03 — Getting Started with GitHub
Module 04 — Working with Repositories
Module 05 — Collaborating Using the GitHub Flow
Module 06 — Tracking Issues and Creating Releases
Module 07 — Understanding GitHub Actions
Module 08 — Creating a GitHub Wiki
Module 09 — Working with Social Features
Module 10 — Understanding GitHub Organizations
Module 11 — GitHub Desktop

2026

# Module 02 — Overview of Git and GitHub

## 1. What is Git?

Git is a Source Control System (also called a Version Control System or VCS). It tracks every change made to files over time, recording who changed what and when. Before Git, developers saved files as project_final.js, project_final_v2.js, and so on — a chaotic approach that Git eliminates entirely.

### Git is a Distributed System

There are two types of version control architectures:

- Centralized (e.g., SVN) — one central server holds all code. If it goes down, no one works.
- Distributed (Git) — every developer has a complete copy of the repository and full history on their local machine. The server is just another copy.

Analogy: in a centralized system the bank holds all the money and you have a card. In a distributed system you have the gold at home too.

| Reason | What It Means |
|---|---|
| Fast | All operations (commit, branch, log) happen locally — no network needed, near-instant |
| Disconnected | Work fully offline — commit, branch, view history — sync when internet is available |
| Powerful | Branching, merging, rebasing, cherry-picking — handles complex team workflows |
| Easy | A handful of commands cover 90% of daily use — very low barrier to entry |

Git is free and open source, released under the GNU General Public License v2.

## 2. What is GitHub?

GitHub is a cloud-based hosting service built on top of Git — not just code storage, but a full collaboration and project management platform.

- Code Hosting — browse files, view blame, compare versions visually
- Pull Requests — structured code review and merge workflow
- Issues — built-in bug tracker and task management
- Actions — CI/CD automation triggered by code events
- Wikis, Projects, Discussions — team collaboration tools
- Social Features — follow developers, star repos, fork projects

## 3. Distributed Version Control

Every Git clone is a complete copy (full history included). You can clone a repo including all history, create local branches, push commits to GitHub, and pull changes from it. Even if GitHub went offline every developer keeps a full working copy.

## 4. Git Configuration — Setting Up Identity

| Level | Scope | Where Stored |
|---|---|---|
| --system | All users on the computer | System-wide config file |
| --global | Your user account (most common) | ~/.gitconfig in home folder |
| --local | One specific repository only | .git/config inside the repo |

```
git config --global user.name "Your Name"
git config --global user.email "your@email.com"
git config --global --list            # view all settings
git config user.name                  # check a specific value
```

# Module 03 — Getting Started with GitHub

## 5. The 3 States of a File in Git

| State | What It Means | Area |
|---|---|---|
| Modified | Changed but Git not yet told about it | Working Directory |
| Staged | Marked for inclusion in the next commit | Staging Area |
| Committed | Permanently saved as a snapshot | Local .git Repository |

Analogy — packing for a trip: Modified = clothes on the bed. Staged = clothes in the open suitcase. Committed = suitcase zipped and stored.

## 6. The 4 Areas of Git

- Working Directory — folder where you edit files in any editor
- Staging Area (Index) — preparation zone: choose exactly which changes go in the next commit
- .git Repository (Local) — hidden folder with Git's complete database of every commit and branch
- Remote Repository (GitHub) — cloud copy teams push to and pull from

```
Working Directory  --git add-->  Staging Area  --git commit-->  .git Local
--git push-->  GitHub
GitHub  --git pull-->  Working Directory
```

## 7. Core Git Commands

| Command | What It Does |
|---|---|
| git | Entry point — lists all available commands |
| git config | Configure identity and settings (--global or --local) |
| git init | Turn any folder into a Git repo (creates .git folder) |
| git clone | Download entire remote repo + full history to local machine |
| git add | Move changes from Working Directory to Staging Area |
| git commit | Save staged snapshot permanently to local .git repo |
| git status | Show what is modified, staged, or untracked right now |
| git log | Show full commit history (--oneline, --graph, -5 flags available) |

### Key Command Examples

```
git init                                # initialise current folder
```

```
git clone git@github.com:user/repo.git
git add .                              # stage all changes
git commit -m "Describe the change"
git status
git log --oneline --graph
```

## 8. Connection Options — HTTPS vs SSH

| | HTTPS | SSH |
|---|---|---|
| How it works | Username + Personal Access Token | Cryptographic key pair |
| Daily use | May prompt for credentials repeatedly | Seamless — never prompts after setup |
| Best for | Quick or one-off access | Regular daily development |

### Setting Up SSH

```
ssh-keygen -t ed25519 -C "your@email.com"  # generate key pair
cat ~/.ssh/id_ed25519.pub                   # copy public key
```

Paste the public key at GitHub > Settings > SSH and GPG Keys > New SSH Key.

```
ssh -T git@github.com                       # test the connection
```

## 9. Understanding Repositories — Public vs Private

| | Public | Private |
|---|---|---|
| Visibility | Anyone on the internet can see it | Only you and invited collaborators |
| Use case | Open source, portfolios, learning | Work projects, proprietary code |
| Collaboration | Anyone can fork and contribute via PR | Invite-only |

## 10. Connecting GitHub Locally — Push, Pull, Fetch

| Command | What It Does |
|---|---|
| git push | Upload local commits to GitHub |
| git pull | Download remote commits and auto-merge into current branch |
| git fetch | Download remote commits without merging — review first, then merge manually |

### Core Workflows

```
git clone git@github.com:user/repo.git  # clone
git add . && git commit -m "message"   # stage and commit
git push origin main                    # push to GitHub
```

```
git pull origin main                    # pull latest from GitHub
git remote add origin git@github.com:user/repo.git  # link local to GitHub
git push -u origin main                 # first push (sets default upstream)
```

# Module 04 — Working with Repositories

## 11. Special Repository Files

| File | Purpose |
|---|---|
| README.md | Front page of the repo — rendered by GitHub. Explains what the project does, how to install and use it |
| LICENSE | Defines what others can do with the code. Without it the code is all rights reserved |
| CHANGELOG.md | Curated human-readable history of changes between versions. Follows Keep a Changelog format |
| SUPPORT.md | How to get help. Prevents Issues from being flooded with support questions |
| CONTRIBUTING.md | Guide for contributors. GitHub links to it when someone opens an Issue or PR |
| CONTRIBUTORS.md | Credits everyone who contributed code, docs, design, or ideas |
| CODE_OF_CONDUCT.md | Community behaviour expectations. Most projects use the Contributor Covenant |
| CODEOWNERS | Auto-assigns reviewers to PRs based on which files changed (.github/CODEOWNERS) |

### CODEOWNERS Syntax

```
*.js              @dev-lead           # All JS files -> dev-lead reviews
/docs/            @org/docs-team      # Docs folder -> docs team
/backend/         @org/backend-team   # Backend -> backend team
```

## 12. Repository Features

| Feature | Description |
|---|---|
| Topics | Coloured tags (e.g. python, beginner-friendly) that make the repo discoverable in GitHub search |
| Projects | Built-in Kanban / table / roadmap project management boards linked to issues and PRs |
| Issues | Bug tracker and task system with labels, assignees, milestones, and PR linking |
| Pull Requests | Where code review and merging happens — diffs, inline comments, approvals |
| Insights | Analytics: contributor stats, commit frequency, traffic, code frequency graphs |
| Settings | Rename, change visibility, manage collaborators, set branch protection rules |

## 13. Repository Insights

| Insight | What It Shows |
|---|---|
| Contributors | Who contributed, lines added/removed, when — recognise active contributors |
| Commits | Commit frequency graph (daily/weekly) — shows project activity patterns |
| Traffic | Views, unique visitors, referring sites, most popular content pages |
| Code Frequency | Lines added vs deleted over time — healthy projects show a balance |

# Module 05 — Collaborating Using the GitHub Flow

## 14. Branches — What and Why

A branch is an independent line of development — a lightweight movable pointer to a specific commit. Branches allow parallel work without interference.

| Reason | Without Branches | With Branches |
|---|---|---|
| Feature development | Blocks main for everyone | Develop in isolation, merge when ready |
| Bug fixes | Risk breaking working code | Fix on a branch, test, then merge |
| Experiments | Could break everything | Try freely, discard the branch if it fails |
| Code review | No clean way to review | PR shows exactly what the branch changed |

## 15. Traditional Source Control vs Git Snapshots

**Delta-based (SVN):** stores original file + list of changes. Must replay all deltas to reconstruct old versions. Gets slow with long history.

**Git Snapshots:** complete snapshot of all files at every commit. Unchanged files are stored as pointers. Instant access to any version, cheap branching.

## 16. Branching in GitHub

- Default branch is main — always stable and deployable
- main points to the latest commit and moves forward with every new commit
- Use branches for: features (feature/login), bugs (fix/crash), experiments, releases

### Branch Commands

```
git switch -c feature/login         # create and switch (most common)
git branch                          # list local branches
git branch -a                       # list all including remote
git push -u origin feature/login    # push branch to GitHub
git branch -d feature/login         # delete locally after merging
git push origin --delete feature/login  # delete on GitHub
```

## 17. Branch Strategies

| Strategy | Description | Best For |
|---|---|---|
| Centralized | Everyone commits directly to main — no branches | Very small teams, simple scripts |
| GitFlow | Long-lived branches: main, develop, feature/*, | Scheduled releases, |

| Strategy | Description | Best For |
|---|---|---|
| | release/*, hotfix/* | enterprise apps |
| Forking | Fork the repo to your account, PR back to original | Open source contributions |
| GitHub Flow | Short-lived branches + PR into main — always deployable | Continuous deployment, most teams |

### GitHub Flow Rules

- main is always in a deployable state
- All work happens on short-lived branches
- Everything goes through a Pull Request — code review is mandatory
- Merged branches are deleted immediately

## 18. Merge Conflicts — When and Why

A merge conflict occurs when both branches changed the same part of the same file differently. Git cannot decide which version to keep and asks you to resolve it manually.

```
<<<<<<< HEAD (main)
Hello World
=======
Hello GitHub
>>>>>>> feature/my-branch
```

```
git add filename.txt
git commit -m "Resolve merge conflict"
```

| Diff Tool | Description | Config Command |
|---|---|---|
| KDiff3 | Visual 3-way merge — base, yours, theirs. Great for beginners | git config --global merge.tool kdiff3 |
| vimdiff | Terminal-based. Powerful but requires Vim knowledge | git config --global merge.tool vimdiff |
| VS Code | Accept Current / Accept Incoming UI buttons — most user-friendly | git config --global merge.tool vscode |

## 19. Pull Requests — Templates and Review

### PR Template — .github/pull_request_template.md

```
## Description
What does this PR do?

## Related Issue
```

```
Fixes #(issue number)

## Checklist
- [ ] Tested locally
- [ ] Documentation updated
- [ ] No new warnings
```

| Review Type | When to Use |
|---|---|
| Comment | General feedback — no approval decision yet |
| Approve | Code looks good, ready to merge |
| Request Changes | Issues must be fixed before merging |

| Merge Option | What It Does |
|---|---|
| Merge commit | Keeps all commits + adds a merge commit — full history preserved |
| Squash and merge | Combines all branch commits into one clean commit on main |
| Rebase and merge | Replays commits linearly onto main — no merge commit |

## 20. Forking a Repository

A fork is a personal copy of someone else's repo on your GitHub account. Unlike cloning (local copy), forking copies it to your GitHub account so you can contribute via PR.

| | Fork | Clone |
|---|---|---|
| Where it lives | Your GitHub account (cloud) | Your local machine |
| Connection | Linked — can submit PRs back | No GitHub-level link to source |
| Use case | Contributing to others' repos | Working on repos you own/have access to |

```
git remote add upstream https://github.com/original-owner/repo.git
git pull upstream main   # sync your fork with the original
```

# Module 06 — Tracking Issues and Creating Releases

## 21. Working with Issues

Issues are GitHub's built-in tracking system — the central place for all work, problems, and ideas about a project.

| Type | Description |
|---|---|
| Bug | Something is broken or not working as expected |
| Enhancement | Request to improve existing functionality |
| Task | Work item that needs to get done (not necessarily a bug) |
| Idea | Early-stage thought or proposal for discussion |

### Notifications

- You get notified when you open, comment on, are assigned to, or are @mentioned in an issue
- Configure per repo: Watch, Not watching, or Ignore
- Email and web notifications (bell icon) are both supported

### Association with Pull Requests

Link an issue to a PR using keywords in the PR description:

```
Fixes #42
Closes #42
Resolves #42
```

When the PR merges into the default branch, the linked issue automatically closes. This creates a complete audit trail from problem reported to code that fixed it.

## 22. Labels on Issues and Pull Requests

Labels are coloured tags that categorise, filter, and prioritise issues and PRs. Multiple labels can be applied to one issue.

| Built-in Label | Purpose |
|---|---|
| bug | Something is not working correctly |
| duplicate | This issue or PR already exists elsewhere |
| enhancement | New feature or improvement request |
| help wanted | Extra attention or community help is needed |
| question | Further information is requested |
| wontfix | This will not be worked on — intentional decision |
| good first issue | Good for newcomers and first-time contributors |

| Built-in Label | Purpose |
|---|---|
| invalid | Does not seem right — wrong repo, misunderstanding, etc. |
| documentation | Improvements or additions to documentation needed |

You can also create custom labels with any name and colour to match your team's workflow.

## 23. Creating Issues and Issue Templates

### Creating an Issue (GitHub UI)

- Go to Issues tab > New Issue
- Add a clear Title and detailed Description (steps to reproduce for bugs)
- Use the right panel: assign Assignees, Labels, Milestone, and Project
- Click Submit new issue

### Issue Template — .github/ISSUE_TEMPLATE/bug_report.md

```
---
name: Bug Report
about: Report a bug to help us improve
title: "[BUG] "
labels: bug
---
## Description
A clear description of the bug.

## Steps to Reproduce
1. Go to '...'
2. Click on '...'
3. See error

## Expected vs Actual Behaviour

## Environment
- OS, Browser, Version
```

You can have multiple templates — one for bugs, one for feature requests, one for questions. GitHub shows a selector when someone clicks New Issue.

## 24. Milestones

A milestone is a target point you're working towards — a version, sprint, or deadline. Issues and PRs are grouped under a milestone to track progress toward that goal.

| Feature | Benefit |
|---|---|
| Progress tracking | GitHub shows a progress bar: X of Y issues closed (e.g. 53% complete) |

| Feature | Benefit |
|---|---|
| Grouping | All issues for a version/sprint in one place — easy to see what remains |
| Prioritisation | Focus the team on what matters for the current goal |
| Due dates | Optional deadline that appears in the milestone view |

### Creating a Milestone

- Go to Issues tab > Milestones > New Milestone
- Give it a title (e.g., v2.0 Release, Sprint 3)
- Add an optional due date and description
- Click Create Milestone — then assign issues to it

## 25. Working with Tags

A tag is a permanent pointer to a specific important commit in history — like a bookmark. Unlike branches (which move forward), tags never move. They permanently mark a fixed point — typically a release version.

| Type | Description | When to Use |
|---|---|---|
| Lightweight | Just a name pointing to a commit — nothing more | Quick private markers, temporary use |
| Annotated | Full object with tagger name, email, date, and message. Can be signed | Production releases — recommended |

### Tag Commands

```
git tag                              # list all tags
git tag v1.0                         # lightweight tag on current commit
git tag -a v1.0 -m "Version 1.0"    # annotated tag (recommended)
git tag -a v0.9 abc1234 -m "Beta"  # tag a specific past commit
git show v1.0                        # view tag details
git log --oneline --tags             # see tags in commit history

# Tags are NOT pushed automatically — must push explicitly:
git push origin v1.0                 # push one tag
git push origin --tags              # push ALL tags
```

## 26. Publishing a Release

GitHub Releases are built on top of tags. A release adds a rich page with notes, assets, and binary files for download.

### How to Create a Release

- Go to repo > Releases > Draft a new release
- Choose an existing tag OR create a new tag right here

- Write a Release title (e.g., Version 2.0 — Dark Mode Update)
- Write Release notes — what changed, new features, fixes. GitHub can auto-generate from merged PRs
- Attach binary files — compiled executables, ZIP archives, installers
- Mark as Pre-release (beta/RC) or full release
- Click Publish release

| Release Element | Description |
|---|---|
| Source code archives | GitHub auto-provides .zip and .tar.gz downloads of the tagged source |
| Binary attachments | Compiled files you upload — users download the built app directly |
| Release notes | Changelog for this version shown prominently on the release page |
| Pre-release flag | Mark as beta/RC so users know it is not yet stable |

# Module 07 — Understanding GitHub Actions

## 27. What is GitHub Actions?

GitHub Actions is an automation and CI/CD platform built directly into GitHub. It lets you automatically run tasks in response to events in your repository.

- Event-driven — triggered by things that happen in GitHub (push, PR opened, issue created, scheduled time)
- Lives in your repo — workflow files are in .github/workflows/ alongside your code
- YAML syntax — workflows are written in simple key-value YAML configuration files
- Covers many parts of the SDLC — testing, building, deploying, notifications, security scanning

## 28. Core Components

| Component | Description |
|---|---|
| Workflow | A configurable automated process — a YAML file in .github/workflows/. One repo can have many workflows |
| Event | What triggers the workflow (push, pull_request, schedule, issues, workflow_dispatch, etc.) |
| Job | A set of steps that runs on a single virtual machine (runner). Jobs run in parallel by default |
| Step | An individual task within a job. Runs sequentially. Either a shell command or a pre-built action |
| Action | A reusable unit that performs one specific task. From the GitHub Marketplace or custom-built |
| Runner | The virtual machine that executes jobs — ubuntu-latest, windows-latest, or macos-latest |

### The Hierarchy

```
Workflow
  └── Job (runs on ubuntu-latest)
        └── Step 1: checkout code          (uses: actions/checkout@v4)
        └── Step 2: install dependencies   (run: npm install)
        └── Step 3: run tests              (run: npm test)
```

## 29. Creating a Workflow in YAML

```yaml
name: CI Pipeline                    # shown in GitHub Actions UI

on:                                  # TRIGGER
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]
```

```
jobs:
  build-and-test:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Set up Node.js
        uses: actions/setup-node@v4
        with:
          node-version: '20'

      - name: Install dependencies
        run: npm install

      - name: Run tests
        run: npm test
```

## 30. Types of Events

| Event Type | Examples |
|---|---|
| GitHub Events | push, pull_request, issues, issue_comment, release, create, delete |
| Scheduled (Cron) | cron: '0 9 * * 1' = every Monday 9am UTC |
| Manual (webhook) | workflow_dispatch — triggered by button click in GitHub UI |
| Combined | Multiple triggers in one workflow (push + schedule + manual) |

### Scheduled Cron Syntax

```
on:
  schedule:
    - cron: '0 9 * * 1'    # Every Monday at 9am UTC
    - cron: '0 0 * * *'    # Every day at midnight
```

Cron format: minute hour day-of-month month day-of-week

## 31. Types of Workflows

| Type | Purpose | Examples |
|---|---|---|
| Automation | Repository management — not code | Auto-label issues, close stale issues, welcome first contributors |
| CI (Continuous Integration) | Code quality on every push/PR | Run tests, linting, code coverage, security scans |
| CD (Continuous Deployment) | Deploy after tests pass | Deploy to staging/production, publish to npm or PyPI |

## 32. Example — Auto-Label New Issues

```
name: Auto-label new issues

on:
  issues:
    types: [opened]

jobs:
  label:
    runs-on: ubuntu-latest
    steps:
      - name: Add 'needs triage' label
        uses: actions/github-script@v7
        with:
          script: |
            github.rest.issues.addLabels({
              issue_number: context.issue.number,
              owner: context.repo.owner,
              repo: context.repo.repo,
              labels: ['needs triage']
            })
```

## 33. CI Using Templates

GitHub provides starter workflow templates — no need to write CI from scratch.

- Go to repo > Actions tab > GitHub suggests templates based on your code language
- Click Configure on any template — customise the YAML and commit
- Available templates: Node.js, Python, Java (Maven/Gradle), Docker, Deploy to Azure/AWS/GCP

# Module 08 — Creating a GitHub Wiki

## 34. What is a GitHub Wiki?

A Wiki is a dedicated documentation space attached to your repository — separate from code files. It is designed for long-lived, structured documentation about how to use, install, configure, and contribute to your project.

| | README.md | Wiki |
|---|---|---|
| Location | Root of your repo — part of the code | Separate tab — its own dedicated space |
| Best for | Quick intro and getting started | Comprehensive, multi-page documentation |
| Structure | Single file | Multiple linked pages |
| Editing | Via PR like any code file | Directly in the GitHub web UI |
| Offline access | Part of regular git clone | Clone separately as its own git repo |

Documentation is vital — well-documented projects attract more contributors and users. A wiki is a long-living document on how to use the project, available both online and offline.

## 35. Setting Up and Working with Your Wiki

### Enable the Wiki

- Go to repo > Settings > Features > check the Wikis checkbox
- Click the Wiki tab that now appears in the repo navigation

### Adding Pages

- Click New Page in the wiki
- Give it a title (this becomes the URL slug)
- Write content in Markdown — same syntax as README
- Click Save Page

### Linking Between Wiki Pages

```
[[Page Name]]              # link to another wiki page
[[Installation Guide]]     # example
```

### Offline Access — Clone Your Wiki

Every wiki is a separate Git repo you can clone, edit locally, and push:

```
git clone https://github.com/username/repo.wiki.git
```

## Recommended Wiki Page Structure

| Page | Contents |
|---|---|
| Home | Landing page — overview of the wiki and links to main sections |
| Installation | Detailed step-by-step setup instructions for all platforms |
| Configuration | All configuration options explained with examples |
| API Reference | Endpoints, parameters, responses (if the project has an API) |
| FAQ | Frequently asked questions and common issues |
| Contributing | More detailed contribution guide than CONTRIBUTING.md |

# Module 09 — Working with Social Features

## 36. Gists

A Gist is a simple way to share a single file or code snippet without creating a full repository. It is backed by Git so it has full version history, supports comments, and can be forked.

| | Public Gist | Secret Gist |
|---|---|---|
| Discoverability | Searchable on GitHub, shows on your profile | Not searchable — only accessible via direct link |
| Privacy | Fully public | Anyone with the link can view (not truly private) |
| Use case | Sharing useful snippets broadly | Work-in-progress or sensitive content |

### How to Create a Gist

- Go to gist.github.com (or click your avatar > Your Gists)
- Add an optional description
- Add a filename with extension (.py, .js, .md, etc.)
- Paste or type your content
- Choose Create public gist or Create secret gist

Gists can contain multiple files and support starring, forking, and commenting.

## 37. GitHub Projects — Project Boards

GitHub Projects is a built-in project management tool — a flexible, customisable board for planning and tracking work, natively linked to your issues and PRs.

| View | Description |
|---|---|
| Board view | Kanban-style columns: To Do, In Progress, Done — drag and drop cards |
| Table view | Spreadsheet-like view of all items with custom fields |
| Roadmap view | Timeline/Gantt chart view for planning across time |

### Key Features

- Custom fields — add Priority, Status, Sprint, Size, or any field you need
- Automation — auto-move cards when issues/PRs are opened, merged, or closed
- Linked to issues and PRs — cards update automatically as work progresses
- Cross-repo — an organization project can pull issues from multiple repos

### How to Create a Project Board

- Go to repo > Projects tab > New Project (or via your profile > Projects)
- Choose a template: Basic Kanban, Bug Tracker, Feature planning, Scrum, or blank
- Name your project and click Create
- Add items: link existing issues/PRs or create draft cards directly
- Customise columns, fields, and views to match your workflow

## 38. GitHub Pages — Publishing a Site

GitHub Pages turns your repository into a free, publicly hosted static website. Push HTML/CSS/JS (or Markdown) to a branch and GitHub automatically serves it as a live site.

| Use Case | Description |
|---|---|
| Project documentation | A full docs site for your open source project |
| Personal portfolio | Showcase your work at yourusername.github.io |
| Blog | Using Jekyll (built-in static site generator) with Markdown posts |
| Course notes or tutorials | Publish learning materials as a formatted website |
| Marketing pages | Landing page for your open source project or product |

### How to Enable GitHub Pages

- Go to repo > Settings > Pages
- Under Source, choose a branch (main or gh-pages) and folder (/root or /docs)
- Click Save — GitHub builds and deploys your site
- URL appears in Settings: https://yourusername.github.io/repo-name/

GitHub Pages natively supports Jekyll — just add Markdown files and a _config.yml and GitHub builds the full HTML site automatically with no build step required.

# Module 10 — Understanding GitHub Organizations

## 39. What is a GitHub Organization?

An Organization is a shared account on GitHub that multiple people can belong to. Repos live under the org name (github.com/orgname/repo) rather than an individual's username.

| | Personal Account | Organization |
|---|---|---|
| Repo ownership | Owned by you personally | Owned by the org — survives member changes |
| Members | Just you | Many people with different roles and permissions |
| Billing | Personal plan | One subscription covers all members |
| Best for | Individual projects | Companies, teams, open source projects |

### Why Use an Organization?

- Centralized ownership — repos stay with the org even if a member leaves
- Team management — group people into teams with specific permission levels per repo
- Billing in one place — one subscription covers everyone
- Branded presence — your company or project has its own GitHub identity
- Fine-grained access control — exactly who can see and do what

## 40. Working with Teams

Teams are groups within an organization. You assign teams to repositories with specific permission levels rather than managing individual users one by one.

| Permission Level | What They Can Do |
|---|---|
| Read | View and clone repos — good for stakeholders, clients, or read-only access |
| Triage | Manage issues and PRs without push access — good for project managers |
| Write | Push code, manage issues and PRs — standard for developers |
| Maintain | Manage repo settings without sensitive actions — team leads |
| Admin | Full access including settings, webhooks, and deletion |

### How Teams Work

- Create a team (e.g., frontend-team, backend-team, docs-team)
- Add org members to the team
- Give the team access to specific repos at a specific permission level
- @mention entire teams in issues/PRs: @orgname/frontend-team

- Teams can have nested sub-teams for large organisations

## Creating a Team

- Go to your org > Teams > New Team
- Give it a name and description
- Set visibility: Visible (all org members see it) or Secret (members only)
- Add members to the team
- Go to Repositories within the team > Add repository > set permission level

# Module 11 — GitHub Desktop

## 41. What is GitHub Desktop?

GitHub Desktop is a free, cross-platform GUI application from GitHub that lets you work with Git and GitHub repositories using a visual interface — without typing any terminal commands.

| Who It Is For | Description |
|---|---|
| Developers | Those who prefer a visual interface for day-to-day Git work |
| Beginners | People still learning Git who benefit from a visual representation |
| Non-developers | Designers, writers, project managers who need to contribute to repos |
| All skill levels | Situations where a GUI is faster — visualising diffs, resolving conflicts |

## 42. What GitHub Desktop Can Do

| Feature | Description |
|---|---|
| Clone | Clone any repository from GitHub with a single button click |
| See changes | All modified files listed visually with side-by-side diff view |
| Stage/unstage | Checkboxes to include or exclude specific changes per commit |
| Commit | Write summary and description, commit with one click |
| Push and pull | Sync with GitHub using one click — no command needed |
| Branches | Create, switch, and delete branches from a dropdown menu |
| Merge conflicts | Visual conflict resolution — see both versions clearly |
| Open PRs | Takes you directly to GitHub to open a PR from the current branch |
| View history | Visual commit log with full diffs for every commit |

## 43. GitHub Desktop vs Command Line

| | GitHub Desktop | Command Line |
|---|---|---|
| Ease | Very easy — visual and intuitive for all users | Steeper learning curve initially |
| Power | Covers 90%+ of daily Git tasks visually | Full Git power — every command available |
| Speed for experts | Fast for visual tasks and reviewing changes | Faster for power users and automation |
| Automation | Not suitable for scripting | Excellent for CI/CD scripts and automation |

| | GitHub Desktop | Command Line |
|---|---|---|
| Best for | Getting started, visual review, conflict resolution | Advanced operations, scripting, CI/CD |

GitHub Desktop and the command line are complementary — many professionals use both: Desktop for day-to-day work and the terminal for advanced operations and automation.